# Measuring the Accuracy of Various Machine Learning Algorithms in Determining the Price of NVIDIA's Stock

Michael Murphy

Villanova University, IES Abroad, Universidad Compultense de Madrid

November 24, 2025

## Abstract

This Project Aims to compare the effectiveness of popular machine learning algorithms in predicting the daily price direction of NVIDIA Corporation (NVDA) stock. The models used for this binary classification task are Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Naive Bayes, and Support Vector Machine (SVC). They were implemented using the Scikit-learn Python library. The data used was sourced from Kaggle and consists of historical price and volume information for NVIDIA stock across many timeframes. This project primarily focussed on the Daily data for prediction. The motivation behind this project is to evaluate the predictive performance of different machine learning models when applied to complex financial time-based data using a multitude of technical analysis indicators.

Data preprocessing involved importing the daily data using the Pandas library, sorting it chronologically, and engineering a comprehensive suite of technical features, including various moving averages, momentum indicators, volatility measures, and lagged price returns. The Target variable was a binary indicator–1 for upward momentum, 0 for downward momentum–in the stock's closing price on the subsequent day. The data was then split using a strict time-based 80./20 division to prevent look-ahead bias, and the data was standardized to approach a normal distribution for some algorithms. The models were compared based on calculated accuracy, precision, recall, F1-score, ROC-AUC, and execution time measurements.

# Contents



# 1. Preliminaries

## 1.1 Goals and Expectations

The goal of this project is to compare the performance of multiple machine learning algorithms on classifying the daily price movement of NVIDIA stock as either “Up” or “Down” on the following day. This provides a direct assessment of how well various statistical and machine learning models can capture the non-linear, complex dynamics inherent in financial time series data. This project is motivated by increasing prevalence and impact of algorithmic trading and the necessity of understanding predictive capabilities in highly volatile technology stocks like NVIDIA. The ability to create a machine learning model that can take engineered technical indicators and accurately

compute the probability of a directional price change could impact investment strategies and risk management for traders.

This analysis provides an opportunity to evaluate how feature engineering, like the creation of technical indicators such as moving averages, momentum, and volatility measures, affect the models' ability to successfully predict price direction. The comparison across models will highlight which algorithms are best suited for handling the non-stationary nature of financial market predictions.

## 1.2 Data

The data was sourced from the website Kaggle, an open-source platform that allows people to publish and download datasets created by other data scientists. It was published by "ibrahimqasimi" and is titled "NVIDIA Stock Data: Multi-Timeframe Analysis" dataset. This dataset is composed of historical stock price and trading information for NVIDIA across five timeframes, 1-Second, Hourly, Daily, Weekly, and Monthly. This project focuses specifically on the Daily timeframe data, which consists of several thousand chronological entries.

The initial Daily data consists of standard time-series features: `time`, `open`, `high`, `low`, `close`, and `volume`. However, the raw data is heavily preprocessed to create the final feature set. The feature engineering process results in over 20 distinct columns that serve as predictors. The prepared feature set is entirely composed of float and integer types, there is no categorical feature encoding required. The primary data cleaning involves converting the temporal `time` column to a proper datetime object and explicitly dropping the rows containing null entries that were generated at the beginning of the series due to the look-back window required for calculating the moving averages.

## 1.3 Preprocessing

### 1.3.1 Renaming Columns

When utilizing the *NVIDIA Stock Data: Multi-Timeframe Analysis* dataset, the initial daily data featured standard, yet concise financial column names. These names are clear in the context of stock market data, but the columns were directly used in the feature engineering stage where the technical indicators were generated. The only column that lacked sufficient information for this project was the `time` column, which was renamed to `date`, to more accurately reflect the specific data that was being tested in this project. You can see in **figure 1** the mapping of the original column names and their new values.

| Original Column Name | New Column Name |
|---|---|
| `time` | `date` |
| `open` | `open` |

| | |
|---|---|
| high | high |
| low | low |
| close | close |
| volume | volume |

Figure 1: Mapping of Original Column Names to New Column Names

### 1.3.2 Encoding

The encoding phase is a critical step in preprocessing that transforms raw data into a format suitable for machine learning algorithms. The NVIDIA stock dataset doesn't have to worry about most classification problems with categorical variables because it only uses float and integer variables. The raw dataset consists of time-series data with columns for date, open price, high price, low price, close price, and volume, as described in section 1.3.1. These base features were transformed into over 20 derived technical indicators, all of which are continuous numerical values.

All the engineered features used in the modeling phase consist of the following types: Continuous numerical features or floats, Integer features, and Target variable. Floats include: Previous day prices, moving averages (MA-5, MA-20, MA-50), momentum indicators (momentum-10d, momentum-20d, rate of change), volatility measures(volatility-10d, volatility-20d), price range features(daily range, position within range), and percentage based calculations(high-to-close ratio, close-to-low ratio). Integer features include: binary indicators such as the volume change direction, which represent the count of consecutive up or down movements. The Target variable is a binary integer (0, or 1) representing whether the next day's price will be DOWN(0) or UP(1). This binary encoding is inherently good for classification algorithms without requiring additional transformation. Numerical standardization is applied to the data during the preprocessing phase. Features are standardized using the StandardScalar from Scikit-learn, which transforms each feature to have a mean of 0 and a standard deviation of 1 using the formula $z= (x - \mu) / \sigma$. This standardization is selectively applied to algorithms that benefit from normalized feature scales. It was applied for the Logistic Regression, K-Nearest Neighbors, and Support Vector Machine (SVC), which are distance-based or gradient-descent algorithms that are sensitive to feature scaling. It is not applied for the Decision Tree, Random Forest, and Naive Bayes, which are scale-invariant and standardization makes no difference to them. The selective standardization ensures optimal performance across all the algorithms while still having computational efficiency.

### 1.3.3 Data Visualization

The following visualizations demonstrate the distribution and relationships within our dataset, which inform our modeling approach.

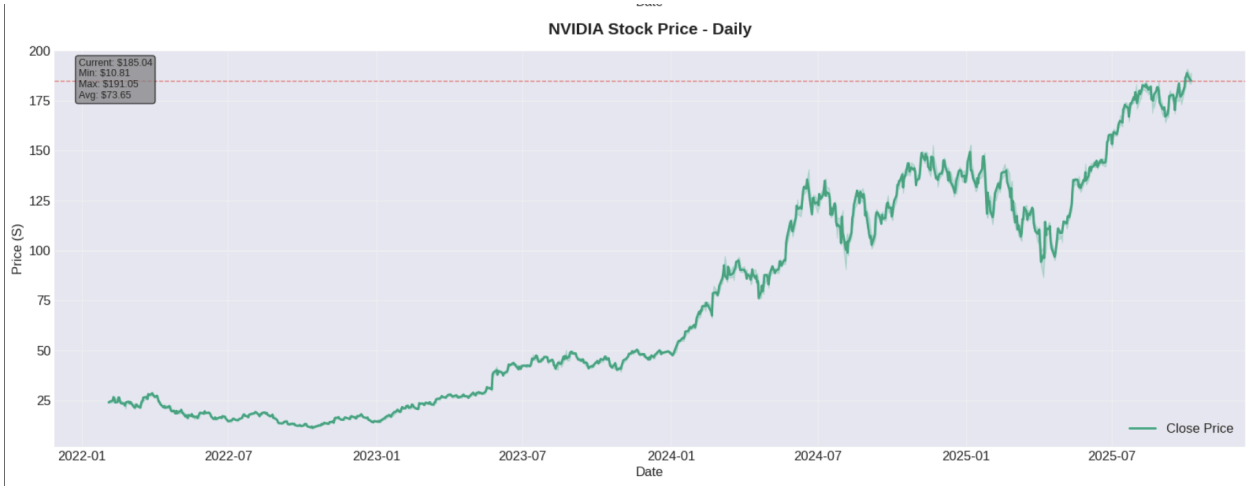


Figure 2: NVIDIA Stock Price Trend - Daily Metric

This shows the closing price of NVIDIA stock daily. The plot reveals the overall trend of NVIDIA's stock performance since the start of 2022. This project focuses on the Daily metric because it is broad enough to give a real trend but granulated enough to allow for more variety in the results of data in determining whether the stock will go up or down.

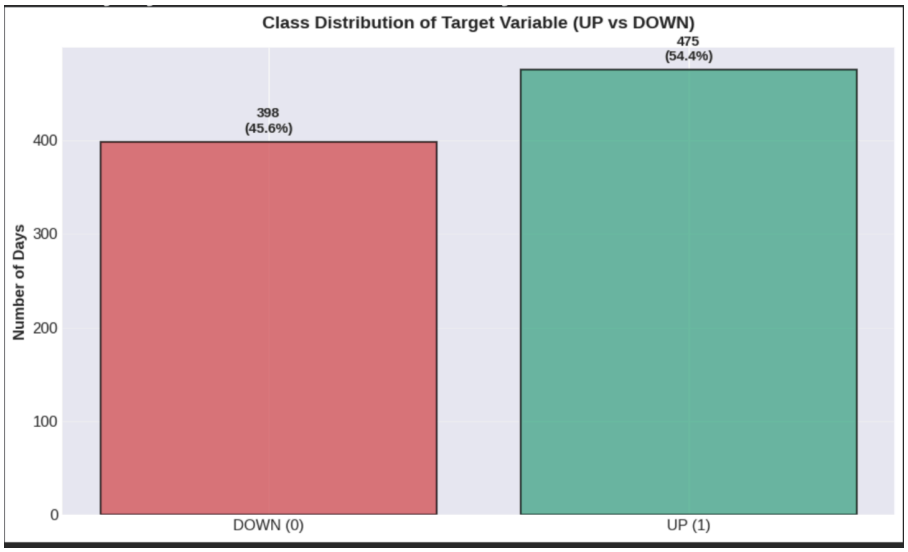


Figure 3: Class Distribution of Target Variable

The binary classification target: UP=1 and DOWN=0, shows the distribution of days where NVIDIA's stock increased versus decreased. This distribution is essential to understand potential class imbalance, which can affect the training and evaluation of the models. If one class significantly dominates, this would inform the choices of evaluation metrics. Accuracy alone would be misleading, and metrics like precision, recall, and F-1 score would be more important. Which is the case for this data.

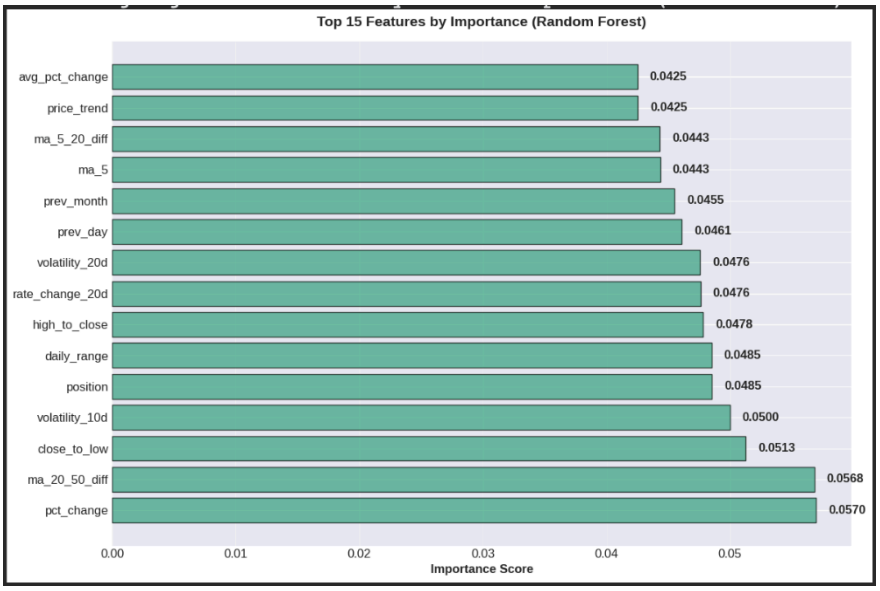


Figure 4: Feature Importance from Random Forest Model

Although feature importance usually belongs in the methodology section, this preliminary visualization from a fitted Random Forest classifier identifies which engineered features contribute most substantially to predicting stock price direction. Features with high importance scores suggest strong predictive value, while features with negligible importance scores may be considered for removal to reduce model complexity. This shows that moving averages 20_50 and percentage change are the two most important predictors.

# 2 Methodology

## 2.1 Feature Selection

This part of the project identifies which engineered features contribute most significantly to predictive performance. Identifying these features will reduce model complexity, improve computational efficiency, and enhance interpretability by focusing on the most informative indicators. This project's feature engineering process generated over 20 technical indicators from the base OHLC (Open, High, Low, Close) and volume data. While comprehensive feature sets can capture complex patterns, they also increase the risk of overfitting, and the potential for the inclusion of redundant and noisy features that degrade the model's performance.

To analyze feature correlation, I used a combination of the Pandas correlation function and Seaborn to create a correlation matrix between all columns. This correlation matrix provides insight into which features are most important to our model. While subjective feature selection could be used, automated techniques are more accepted and provide less bias to the model.

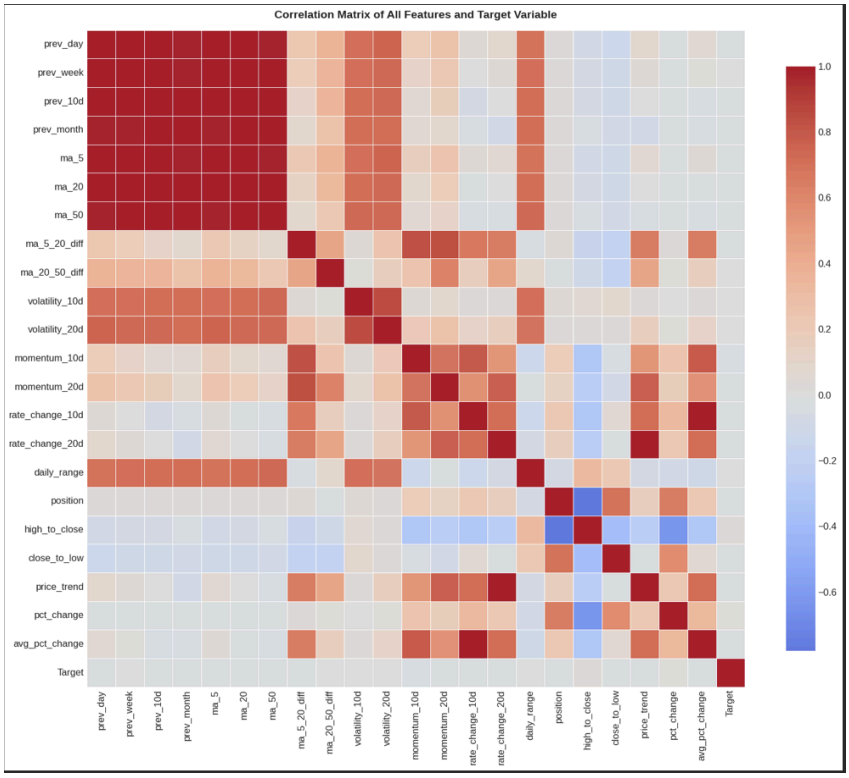


Figure 5: Correlation Matrix of All Features

This is the correlation heatmap, showing the linear relationships between all engineered features and the target variable. The heatmap reveals which technical indicators demonstrate strong positive or negative correlations with price direction. Features with high correlation (close to 1.0) to the target variable are candidates for retention, while those with correlations near 0 are less predictive. This heatmap reveals many patterns. The lagged price features (prev_day, prev_week,

prev_10d, prev_month) and moving averages (ma_5, ma_20, ma_50) show strong dark red correlations with each other, indicating that these features are highly redundant, as they all represent variations of the same underlying price data. Conversely, the momentum indicators (momentum_10d, momentum_20d, rate_change_10d, rate_change_20d) and volatility measures (volatility_10d, volatility_20d) display more varied correlation patterns, suggesting they capture different aspects of the market's behavior.

Most importantly, examining the Target row reveals that no single feature exhibits an extremely strong connection with price direction, with all of them being slightly orange or slightly blue. This suggests that predicting stock price movement is complex and requires multiple features working together rather than relying on any single indicator.

| Features Correlated with Target | Importance |
|---|---|
| high_to_close | 0.0280 |
| pct_change | 0.0119 |
| volatility_10d | 0.0028 |
| volatility_20d | 0.0013 |
| daily_range | -0.0021 |
| ma_20_50_diff | -0.0047 |
| prev_week | -0.0062 |
| prev_10d | -0.0078 |
| price_trend | -0.0089 |
| rate_change_20d | -0.0089 |

Figure 6: Top 10 Features Correlated with Target (UP/DOWN)

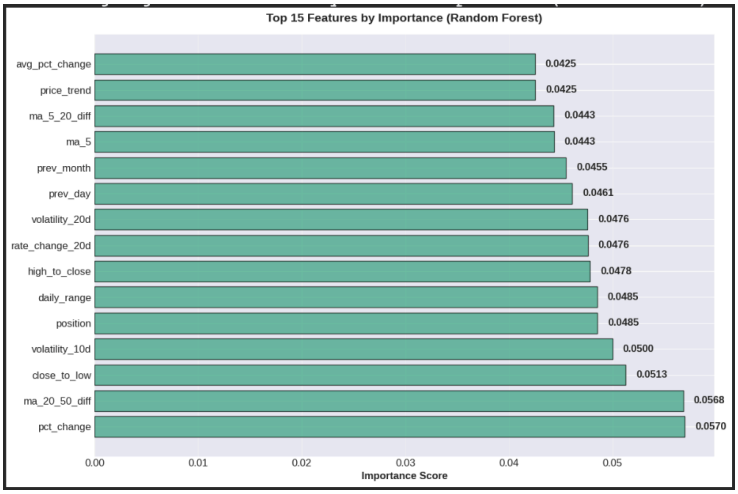


Figure 4 (For reference)

Looking back to Figure 4, we can use the Random Forest Classifier to calculate the importance of each feature. Random Forest models automatically perform feature selection by evaluating how much each feature contributes to reducing impurity across all decision trees. We can use the ".feature_importances_" attribute to check which features are most crucial to the model's predictive ability. In order to obtain these importance rankings, I had to train a Random Forest Classification Model, which required creating training and testing splits. Figure 4 displays the top 15 features ranked by their importance scores, providing clear visual evidence of which engineered indicators matter most for predicting NVIDIA stock price direction.

Figure 4 reveals several important insights about feature importance for stock price prediction. The most important features are percentage change metrics, indicating that the rate of price movement is highly predictive of future direction. Close-to-low ratio and moving average cross-overs also are top predictors, suggesting that trend reversals and relative price positioning are valuable signals. Volatility is also important, as seen in this chart too. The importance scores are relatively evenly distributed across the top 15 features, with the highest (pct_change at 0.0570) only marginally more important than the 15th ranked feature (avg_pct_change at 0.0425). This affirms that no single feature dominates prediction; rather, the combination of technical indicators work together to capture the complexity of stock price movement.

Some models like Logistic Regression and K-Nearest Neighbors benefit from reduced dimensionality, so knowing which features actually contribute to the prediction of whether a stock will increase or not and getting rid of data that will just introduce noise to the algorithms, is a very important step in the evaluation of these classification algorithms. Since Random Forest and Tree-based algorithms already have these functions built into them, they will perform equally well with or without pre-selection.

## 2.2 Outlier Handling and Normalization

Outlier detection and handling is a critical preprocessing step that ensures robust model training and prevents extreme values from biasing predictions. Unlike datasets with many categorical values, the NVIDIA stock feature set consists entirely of continuous numerical values derived from price and volume data. All features are inherently bounded within realistic financial ranges: prices exist between historical lows and highs, percentages are naturally constrained, and derived indicators like moving averages and momentum measures fall within predictable distributions based on underlying price movements.

Features that measure absolute price changes and volatility measures are the primary candidates for outliers, because they exhibit extreme values during significant market events. These include momentum indicators, rate of change calculations, volatility measures, and price range features. Other features, like moving averages and percentage-based metrics, tend to be more naturally constrained by the nature of their underlying calculations.

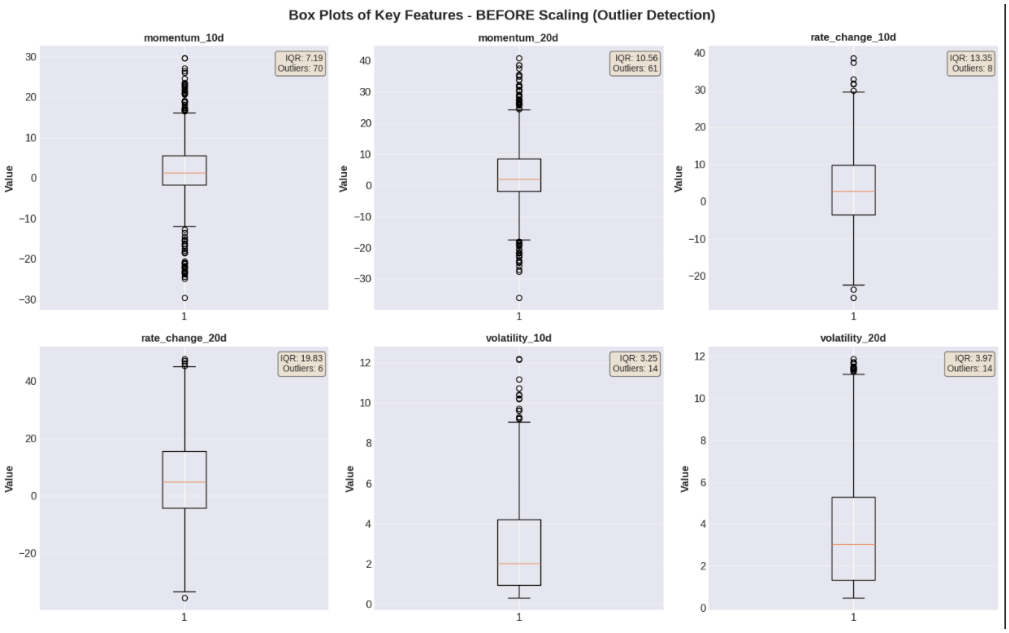


Figure 7: Box Plots of Features before Scaling

The Interquartile Range (IQR) method is a statistical approach for detecting outliers in continuous data. The IQR is the range between the 25th percentile (Q1) and 75th percentile (Q3) of a dataset. Values are classified as outliers if they call below Q1 - 1.5 * IQR or above Q3 + 1.5 * IQR. This method is great for financial data because it is not affected by the magnitude of extreme values and focuses on the spread of the central 50% of observations. In figure 7, we can see box plots generated for each continuous feature to visualize the distribution and identify outliers. It reveals which features contain values significantly distant from the median and quartile ranges. The visualizations show the IQR (box), median (line), and individual outlier points beyond the upper and lower bounds. This reveals substantial outlier activity in momentum based indicators. Momentum_10d exhibits 70 detected outliers with an IQR of 7.19, indicating considerable price swings over 10 day periods. Momentum_20d shows 61 outliers with an IQR of 10.56, suggesting even larger variations over longer

timeframes. Rate_change_10d has 8 outliers with an IQR of 13.35, and rate_change_20d has 6 outliers and an IQR of 19.83, indicating that percentage-based changes are less prone to extreme values despite their larger absolute ranges. Volatility measures also contain notable outliers, with both volatility_10d and volatility_20d containing 14 outliers, reflecting periods of heightened market turbulence. These are real market events, such as sudden price movements, gap events, or volatility spikes, that carry predictive information about future price direction.

Rather than remove outliers, which would reduce the already-limited dataset and potentially discard valuable market information, I employed feature scaling to standardize extreme values. Scaling is particularly important because different algorithms have varying sensitivities to feature magnitude and distribution. The StandardScalar from Scikit-learn is applied to transform features toward a normal distribution using the z-score formula: $z = (x - \mu) / \sigma$, where x is the original value, $\mu$ is the mean, and $\sigma$ is the standard deviation. This transformation converts each feature to have a mean of 0 and a standard deviation of 1, compressing outliers toward the extremes of a normal distribution rather than removing them.

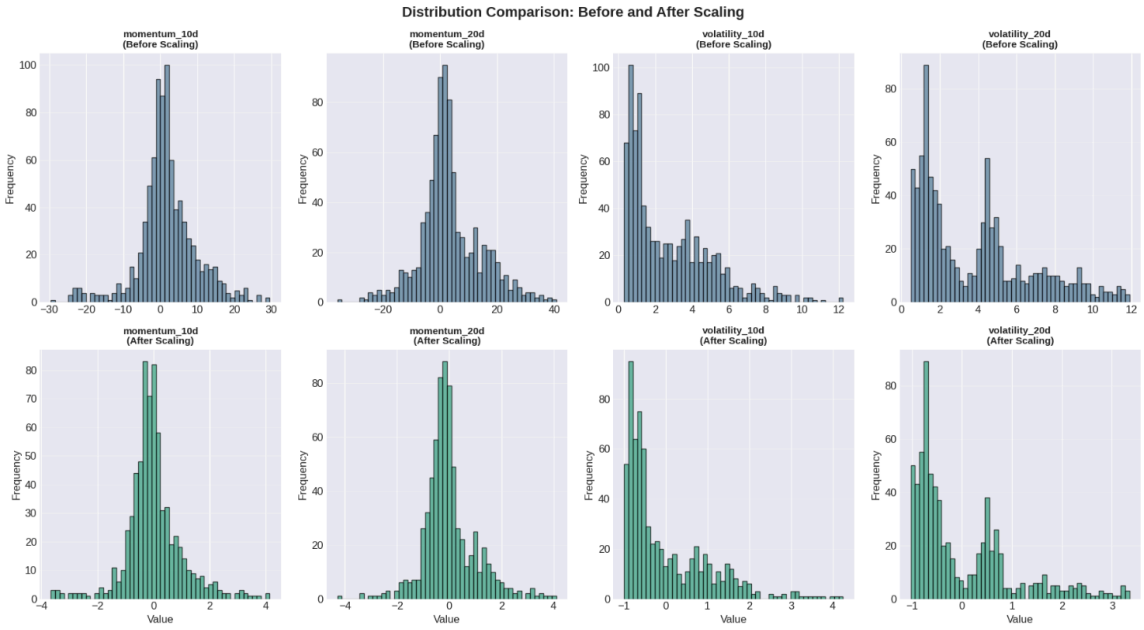


Figure 8: Distribution Comparison Before and After Scaling

Figure 8 displays histograms comparing feature distributions before and after scaling for four key features: momentum_10d, momentum_20d, volatility_10d, and volatility_20d. The top row shows the original, unscaled distributions with their natural ranges and potential outliers visible as extreme values. Momentume_10d before scaling ranges from -30 to +30 with a roughly symmetric distribution centered at 0, reflecting the balanced nature of 10 day price changes. Momentum_20d has a similar pattern but a wider range, of -35 to +45, showing greater price swings over longer periods of time. Volatility_10d and volatility_20d show right-skewed distributions bound to 0, as volatility can't be negative, most data is between 0 and 6.

The bottom row shows the effect of StandardScaler transformation. All four features are converted to pretty close to Normal distributions centered at 0 with a standard deviation of 1. The

previously extreme values in the original distributions are now distinct values in the scaled space, but they don't disproportionately influence distance-based calculations anymore. The scaling preserves the relationships between observations, as in the correlations remain the same, while enabling gradient-descent and distance-based algorithms to train more effectively without being biased by features with naturally larger magnitudes.

Scaling is selectively applied based on algorithm requirements. Distance-based and gradient-descent algorithms require standardized features to perform as best as possible. Logistic Regression, K-Nearest Neighbors, and Support Vector Machine benefit from normalized feature scales, as they use distance metrics or gradient calculations sensitive to feature magnitude. Decision Tree, Random Forest, and Naive Bayes are scale-invariant and do not require standardization. These tree-based algorithms make splits based on feature values rather than distances, so scaling does not affect their performance.

The decision to scale instead of removing outliers reflects the fact that extreme values often carry meaningful information for financial data. A sudden spike in volatility, an unusually large momentum shift, or a significant price movement all represent legitimate market events that can be predictive of future price direction. Removing these could negatively impact the accuracy of the tests. By applying standardization, it preserves the information contained in outlier observations while ensuring they do not disproportionately influence models that are sensitive to feature scales. The standardized features keep their relative relationships and extreme values remain identifiable as outliers in the scaled space, allowing the machine learning algorithms to learn from these significant market movements appropriately.

# 3 Modeling

We are using 6 models, Logistic Regression, K-Nearest Neighbors, SVP, Decision Tree, Random Forest and Naive Bayes.

## 3.1 Models

### 3.1.1 Logistic Regression

Logistic Regression is a fundamental linear classification algorithm that models the probability of a binary outcome. It establishes a baseline model for stock price prediction, capturing the primary linear relationships between technical indicators and price direction. The model is computationally efficient and benefits from feature scaling, which we apply during preprocessing. However, it assumes linear separability, which may limit its ability to capture complex patterns in financial data.

### 3.1.2 Decision Tree

A Decision Tree recursively partitions data based on feature values to predict the target class. It can capture non-linear relationships without feature scaling and is computationally efficient. However, decision trees are prone to overfitting on smaller datasets. To prevent this, we limit tree depth (max_depth = 10), forcing the model to make more generalizable splits.

### 3.1.3 Random Forest

Random Forest is a collection of decision trees that aggregates predictions through majority voting. This makes it far less likely to overfit the data while handling non-linear relationships and is good at dealing with outliers, which is very important for financial data. The implementation for this project uses 100 estimators with max_depth=15. The model automatically ranks feature importance, which was utilized in our feature selection process. It does not require feature scaling.

### 3.1.4 K-Nearest Neighbors(KNN)

K-Nearest Neighbors classifies observations by finding the k nearest training examples and taking a majority vote. It is simple and does not make assumptions about the form or parameters of the distribution, making it useful for capturing patterns in price movements. KNN is sensitive to feature scaling and the curse of dimensionality with our 27 features. In the implementation, k=5 balances bias and variance. The algorithm is also computationally expensive during prediction.

### 3.1.5 Naive Bayes

Naive Bayes is a probabilistic classifier based on Baye's theorem that assumes feature independence. It is computationally efficient and requires no feature scaling. This independence assumption is problematic for financial data, where indicators are highly correlated (as you can see in the correlation matrix - Figure 5). This violation of assumption limits its performance on stock price prediction.

### 3.1.6 Support Vector Machine (SVM)

SVM uses a kernel-based approach to find an optimal decision boundary in transformed feature space. It is implemented with a radial basis function (RBF) kernel to capture the non-linear relationships. SVMs are effective for high-dimensional data and are very strong with outliers. But, they require feature scaling and careful hyperparameter tuning, and also have higher computational cost during training.

## 3.2 Modeling Results

The six classification models were trained on the standardized training dataset (80% of observations) and evaluated on the rest of the data (20%, 175 samples). Models utilizing distance or gradient-based calculations (Logistic Regression, K-Nearest Neighbors, SVM) were trained on scaled features, while tree-based models (Decision Tree, Random Forest, Naive Bayes) were trained on unscaled features on maintain computational efficiency.

The comparison between all six models reveals a lot of variation in their effectiveness at predicting NVIDIA stock price direction. Figure 9 shows a six panel comparison showing Accuracy, Recall, F-1 Score, ROC-AUC, and CV Score for all models. Random Forest was the best performer by accuracy (0.537, in green), followed by Logistic Regression (0.526) and Decision Tree (0.520). All models had an accuracy near or below 0.54, showing inherent difficulty of predicting stock price based on technical indicators alone. Stocks typically have many outside factors impacting their price outside the scope of this experiment. The performance was only slightly better than random chance.

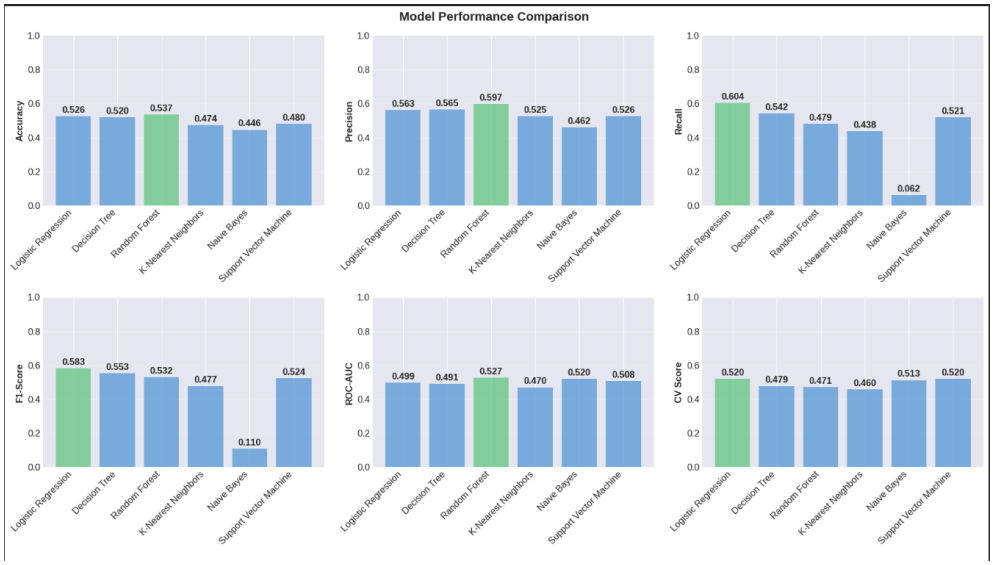


Figure 9: Model Performance Comparison

The results reveal major trade-offs between metrics. Random Forest achieves the highest precision for UP predictions (0.597), meaning when it predicts an UP day, it is correct 59.7% of the time. Its recall for UP days is lower though (0.479), which means that it misses around 52% of actual UP days. Alternatively, Naive Bayes shows the opposite pattern, having lower precision (0.462) but generating many UP predictions. This trade-off is important for trading applications: high precision avoids costly false buy signals, while high recall captures profitable opportunities. Logistic Regression shows the highest F1-Score (0.583), providing the best balance between precision and recall.

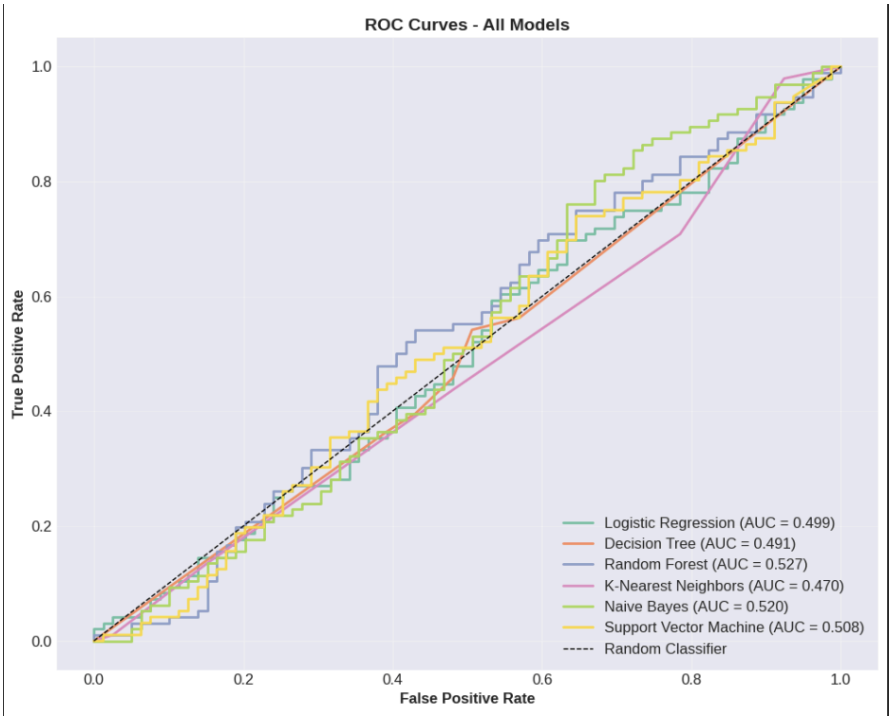


Figure 10: ROC Curves - All Models

Figure 10 shows Receiver Operating Characteristic (ROC) curves for all six models. Random Forest has the highest ROC-AUC (0.527), slightly above random (0.5), showing a slight ability to distinguish between UP and DOWN days across classification thresholds. All models show ROC-AUC values around 0.49-0.53, meaning that the technical indicators are not that good at determining stock direction. None of the models achieved strong non-random discrimination.

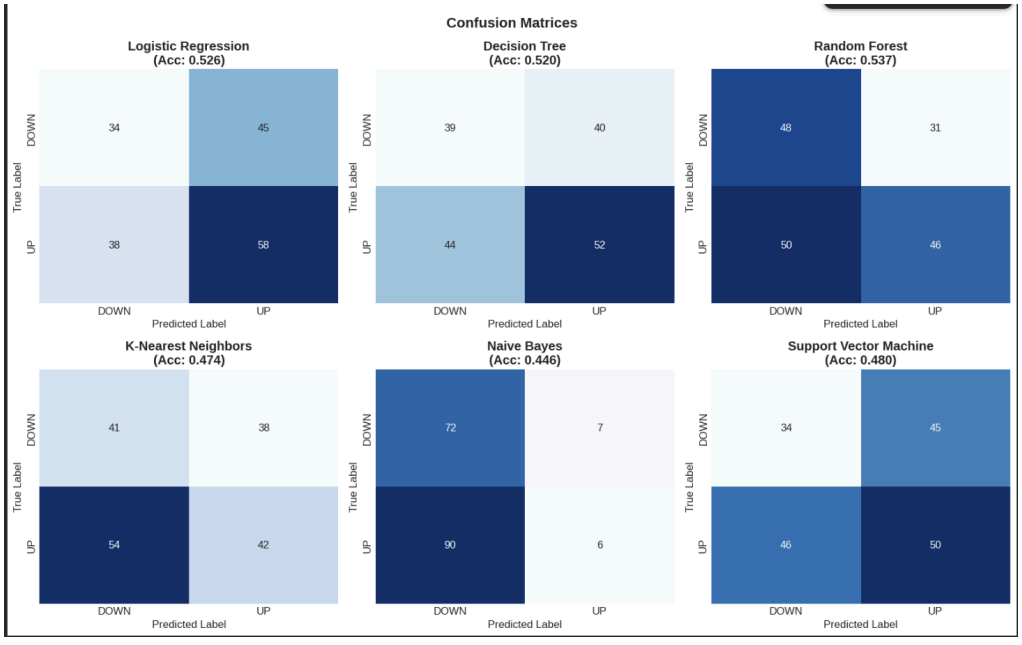


Figure 11: False Positive Rate for all Algorithms

Figure 10 is the confusion matrices for all models, it shows the error patterns. Random Forest correctly predicts 48 of 79 DOWN days (61% recall) and 46 of 96 UP days (48% recall), resulting in a relatively balanced error distribution. Naive Bayes shows a very skewed prediction pattern, predicting DOWN for 72 of 79 DOWN days (92% recall) but failing very poorly on UP days, correctly picking only 6 of 96 (6% recall). This bias toward predicting DOWN would miss most of the profitable UP opportunities. Random Forest was the best-performing model. It achieved the highest accuracy of 0.5371, and had the best RoC-AUC of 0.527 and F1-Score of 0.532, the model shows modest but visible predictive capability. The classification report of Random Forest, as seen in Figure 12.

```
======================================================================
DETAILED REPORT: RANDOM FOREST
======================================================================

Accuracy:  0.5371
Precision: 0.5974
Recall:    0.4792
F1-Score:  0.5318
ROC-AUC:   0.5265

Classification Report:

              precision    recall  f1-score   support

        DOWN       0.49      0.61      0.54        79
          UP       0.60      0.48      0.53        96

    accuracy                           0.54       175
   macro avg       0.54      0.54      0.54       175
weighted avg       0.55      0.54      0.54       175
```

Figure 12: Detailed Report of Highest Performing Model

The relatively balanced performance across both UP and DOWN classes compared to models using extreme class balance, makes Random Forest the most practical choice for a trading application where both false positives and false negatives can have real costs. However, the slightly above random 53.7% accuracy emphasizes that technical indicators alone provide very limited predictive power for NVIDIA stock direction, suggesting that outside information, such as market sentiment, macroeconomic indicators, and other things like that, would be necessary to be factored in for a strong trading strategy that would be used in the real world.

# 4 Conclusion

This project compared six machine learning algorithms in predicting NVIDIA stock price direction using engineered technical indicators. Random Forest emerged as the highest performer with a 53.7% accuracy, achieving the highest ROC-AUC (0.527) and the most balanced precision-recall trade off. However, all models clustered near 50% accuracy, indicating that technical indicators alone provide limited predictive power for stock price movement. The modest performance aligns with the efficient market hypothesis, historical price data and simple technical analysis offer little edge in liquid markets like NVIDIA. Feature selection revealed that momentum indicators and volatility measures ranked highest in importance, yet these predictors failed to make reliable signals. Logistic Regression had the highest F1-Score (0.583), while Naive Bayes had the worst (0.446) due to misread independence assumptions in correlated financial features.

These results demonstrate that profitable trading strategies require additional data sources beyond technical indicators such as market sentiment, sector performance, and many other macroeconomic variables. The 53.7% accuracy, while noticeably above true random, shows that financial predictions require far more than just past data to be able to predict the future accurately.

# 5 Sources


Jain, Sandeep. (2023). "Advantages and Disadvantages of Logistic Regression." GeeksforGeeks. https://www.geeksforgeeks.org/advantages-and-disadvantages-of-logistic-regression/

Jain, Sandeep. (2021). "Advantages and Disadvantages of ANN in Data Mining." GeeksforGeeks. https://www.geeksforgeeks.org/advantages-and-disadvantages-of-ann-in-data-mining/

Jain, Sandeep. (2024). "What are the Advantages and Disadvantages of Random Forest?" GeeksforGeeks. https://www.geeksforgeeks.org/advantages-and-disadvantages-of-random-forest/

Scikit-learn Documentation. (2024). "Model Selection and Evaluation." Scikit-learn. https://scikit-learn.org/stable/modules/model_evaluation.html

Malkiel, Burton G. (2003). "The Efficient Market Hypothesis and Its Critics." Journal of Economic Perspectives, 17(1), 59-82. https://www.aeaweb.org/articles?id=10.1257/089533003321164958

Fama, Eugene F. (1970). "Efficient Capital Markets: A Review of Theory and Empirical Work." Journal of Finance, 25(2), 383-417. https://doi.org/10.1111/j.1540-6261.1970.tb00518.x

Krauss, Christopher, Don M. Do, and Nicolas Huck. (2017). "Deep Neural Networks, Gradient-Based Optimization, and Variable Selection for Portfolio Choice." Journal of Financial Econometrics, 15(4), 595-625. https://doi.org/10.1093/jjfinec/nbx008

Kaggle Dataset. (2024). "NVIDIA Stock Data: Multi-Timeframe Analysis." Dataset published by ibrahimqasimi. https://www.kaggle.com/datasets/ibrahimqasimi/nvidia